UNIT 1

Software and Software Engineering

Software and Software Engineering

- Software engineering stands for the term made of **two** words, S**oftware** and E**ngineering.**
- Software is a **complete package**, not just code.
- It includes:
  - **Program (code)**
  - **Libraries**
  - **Documentation**
- A program alone cannot be called full software without support files and documentation.
- Software is created to **solve real-world problems**.

Software and Software Engineering

Program:

- A program is:
  - A set of **executable instructions**
  - Written in a programming language
- It performs a **specific computational task**
- Example:
  - Calculating numbers
  - Sorting data

Libraries

- Pre-written code that supports the program
- Helps reduce effort and reuse functions

Documentation

- User manuals
- Technical guides
- Instructions for use and maintenance

Software and Software Engineering

- **Engineering** is all about developing products, using well-defined, scientific principles and methods.
- **Software engineering** is an engineering branch associated with development of software product using well-defined scientific principles, methods and procedures.
- The result of software engineering is a **software product** that is:
  - **Efficient** -> works fast and uses fewer resources
  - **Reliable** -> gives correct output and works without failure
  - **Maintainable** -> easy to update and fix
- Software, when made for a specific requirement is called **software product.**

Software and Software Engineering

- **IEEE** defines software engineering as:

1. The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software.
2. It is also the study of methods and approaches used in software development.

- **Fritz Bauer**, a German computer scientist, defines software engineering as:

1. Software engineering is the establishment and use of sound engineering principles to obtain economically software that is reliable and work efficiently on real machines.

Advantage of using software engineering

- The advantages of using software engineering for developing software are:
  - **Improved quality**
  - **Improved requirement specification**
  - **Improved cost and schedule estimates**
  - **Better use of automated tools and techniques.**
  - **Better maintenance of delivered software**
  - **Well defined process**
  - **Improved reliability**
  - **Improved productivity**
  - **Less defects in final processes**

Nature of Software

1. Software as a Product

- Software itself is a **complete product** that can be used by users.
- It is developed to **solve specific problems or perform tasks**.
- As a product, software:
  - Processes data
  - Produces meaningful output
  - Helps users make decisions
- It acts as an **information transformer**, meaning:
  - It takes raw data as input
  - Converts it into useful information
  - Presents it in a readable form (text, reports, graphs, etc.)
- Example:
  - A banking application processes transactions and shows account details.
  - A school management system generates student reports.

Nature of Software

2. Software as a Vehicle for Delivering a Product

- Software also acts as a platform or medium that helps deliver other products and services.
- It does not just produce information but also supports other systems and applications.
- It helps in:
  - **Controlling computer systems**
    - Example: Operating systems manage hardware and software resources
  - **Communication of information**
    - Example: Internet, messaging apps, email systems
  - **Creating other software**
    - Example: Programming tools, compilers, development environments
- In simple words:
- Software is like a carrier that delivers services and connects systems together.

Nature of Software

3. Information as the Main Product

- The most important output of software today is **information**.
- Software:
  - Collects data from users or systems
  - Processes and organizes it
  - Converts it into useful knowledge
- This information is used in:
  - Business decisions
  - Education systems
  - Healthcare systems
  - Communication networks

Characteristics of Software

- Software has some unique characteristics that are very different from hardware.

1. **Software is Developed, Not Manufactured**
   - Software is **engineered (developed)**, not manufactured like physical products.
   - In hardware:
     - Products are made in factories.
     - Machines and physical processes are used.
   - In software:
     - It is created using **design, coding, and testing**.

Characteristics of Software

- Even though both need **good design**, they are different in many ways:
  - Hardware manufacturing can introduce physical defects.
  - Software does not have physical manufacturing defects but may have coding/design errors.
- Software quality depends mainly on:
  - Good design
  - Proper planning
  - Skilled developers
- Software development cost is mainly in **engineering (design and development)**, not production. Therefore, software projects cannot be managed like factory production.

Characteristics of Software

2. Software Does Not "Wear Out"

- Hardware systems wear out over time due to physical reasons.
- Hardware wears out due to:
  - Dust
  - Heat
  - Vibration
  - Aging of components
- Software is different because:
  - It has no physical parts
  - It does not get damaged by environment
- However:
  - Software can become **outdated or faulty due to changes or bugs**
  - But it does not "wear out" like machines

- Simple meaning: Software does not degrade physically over time.

Characteristics of Software

3. Software is Mostly Custom Built

- Most software is still developed for specific needs (custom-built).
- However, modern software development is moving toward reuse of components.

Software Components

- A software component is:
  - A small reusable part of software
  - Designed to be used in different programs
- Good components:
  - Can be reused multiple times
  - Save time and effort

Software Application Domains

- Software is used in many different fields. These fields are called **software application domains**. Each domain has different types of software:
  - System Software
  - Application Software
  - Engineering and scientific software
  - Embedded Software
  - Product line
  - WebApps / Web Application
  - Artificial Intelligence Software (AI based software)

Software Application Domains

1. System Software

- System software is a set of programs that helps run and manage other software.
- It acts as a base for application software.
- It controls and manages computer hardware and system resources.
- **Examples:**
  - Operating systems
  - Compilers
  - Editors
  - File management systems

Software Application Domains

2. Application Software

- Application software is designed to **solve specific user or business problems**.
- It is used directly by users to perform tasks.
- It can work in real time to control business operations.
- **Examples:**
  - Point-of-sale systems (billing systems in shops)
  - Manufacturing control systems
  - Banking applications

Software Application Domains

3. Engineering and Scientific Software

- This type of software deals with complex calculations and numerical data.
- It is often called “number-crunching software”.
- Used in scientific research and engineering analysis.
- **Examples:**
  - Space shuttle calculations
  - Automotive stress testing
  - Astronomy simulations
  - Molecular biology research
  - Volcanology studies

Software Application Domains

4. Embedded Software

- Embedded software is built **inside a device or system**.
- It controls the **functioning of hardware products**.
- It can perform:
  - Simple control tasks
  - Complex system control tasks
- **Examples:**
  - Microwave oven control system (keypad functions)
  - Car systems (fuel control, braking system, dashboard display)
  - Washing machines

Software Application Domains

5. Product-Line Software

- Product-line software is designed for **reuse across many users or customers**.
- It is developed once and used in multiple systems.
- It targets:
  - Small specific markets
  - Large general markets
- **Examples:**
  - Word processors
  - Spreadsheets
  - Database systems
  - Multimedia tools
  - Financial applications

Software Application Domains

6. Web Applications (WebApps)

- Web applications run on the internet or web browsers.
- They are network-based software systems.
- They can range from simple to very complex systems.
- **Examples:**
  - Websites with information
  - Online shopping systems
  - Social media platforms
  - Online banking systems

Software Application Domains

7. Artificial Intelligence (AI) Software

- AI software solves complex problems using intelligent methods.
- It does not depend only on numerical calculations.
- It uses learning, reasoning, and pattern recognition.
- **Examples:**
  - Robotics systems
  - Expert systems (decision-making software)
  - Voice and image recognition
  - Game-playing programs (like chess engines)

New Software Challenges

- Modern software development faces several new challenges due to advances in technology and global connectivity.

1. **Open-world Computing**
   - Open-world computing means building software systems that work across large and open networks.
   - It allows different types of devices (small and large) to communicate with each other.

- Creating software to allow machines of all sizes to communicate with each other across vast networks (Distributed computing—wireless networks)

New Software Challenges

2. Net sourcing

- Net sourcing means using the internet (web) as a platform for computing and applications.
- In this approach:
  - Software is delivered through the web
  - Users access applications online instead of installing them

- Architecting simple and sophisticated applications that benefit targeted end-user markets worldwide (the Web as a computing engine)

New Software Challenges

3. **Open Source**
   - Open source means software whose source code is freely available to everyone.
   - Users can:
     - View the code
     - Modify the code
     - Improve the code
     - Share it with others
   - It is supported by the software development community.

- Distributing source code for computing applications so customers can make local modifications easily and reliably (“free” source code open to the computing community)

Legacy Software

- Legacy software refers to **old computer programs** that were developed many years ago (often decades ago).
- These systems are still in use even though they are outdated.
- **Characteristics of Legacy Software**
  - Legacy software usually has poor quality design.
  - It is difficult to maintain or update because:
    - The design is not flexible (inextensible design)
    - The code is complex and confusing
    - Documentation is poor or missing
    - Test cases and results are incomplete or unavailable
  - Because of these issues, modifying legacy software becomes **very difficult and risky.**

Why Legacy Software Still Changes Over Time?

Unique nature of webapps

- Web applications have some special characteristics that make them different from normal software systems.

1. **Network Intensiveness**
   - WebApps always work through a **network (Internet or Intranet)**.
   - They must support users connected from different locations.
2. **Concurrency**
   - Many users can use the WebApp at the **same time**.
   - Users may perform different actions simultaneously.
   - The system must handle multiple requests without crashing.
3. **Unpredictable Load**
   - The number of users is **not constant**.
   - It can change suddenly:
     - Few users one day
     - Thousands another day

Unique nature of webapps

4. **Performance**
   - WebApps must respond **quickly**.
   - If the system is slow:
     - Users may leave the application
     - Business may lose customers
5. **Availability**
   - WebApps are expected to be available almost all the time.
   - Common expectation:
     - **24/7/365 access** (any time, any day)
   - Full 100% availability is difficult but expected as much as possible.
6. **Content sensitive**
   - The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp.

Unique nature of webapps

7. Data Driven

- WebApps mainly work with **data and information**.
- They display:
  - Text
  - Images
  - Audio
  - Video
- Often connected to databases for storing and retrieving data.

8. Continuous Evolution

- WebApps are updated **continuously**, not in long fixed versions.
- Changes happen frequently to:
  - Add features
  - Fix issues
  - Improve performance

Unique nature of webapps

9. Security

- WebApps are accessed over networks, so they are **vulnerable to attacks**.
- Security is needed to:
  - Protect user data
  - Prevent unauthorized access
  - Ensure safe transactions

10. Immediacy

- WebApps must be developed and released **very quickly**.
- Time to market is often:
  - Days or weeks
- This is important for competition.

Unique nature of webapps

11. Aesthetics

- Appearance (look and feel) is very important.
- A good design helps:
  - Attract users
  - Improve user experience
- Especially important in:
  - E-commerce
  - Social media
  - Marketing websites

Software Engineering - A Layered Technology

- To build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities
  - **Problem should be understood before software solution is developed**
  - **Design is a pivotal Software Engineering activity**
  - **Software should exhibit high quality**
  - **Software should be maintainable**

Software Engineering - A Layered Technology

- Software engineering is a layered technology. And the layer is bottom-up approach.
  - **Quality Focus (The Bedrock)**
    - The foundation of all software engineering activities.
    - It ensures that every phase aims for high reliability, correctness, and user satisfaction.
  - **Process (The Foundation)**
    - Defines the structure of development—who does what, when, and how.
    - It keeps the project organized and manageable.

Fig. - Software Engineering Layers

Software Engineering - A Layered Technology

- **Methods (The "How-To")**
  - The actual techniques used—requirements analysis, design modeling, coding, testing, etc.
- **Tools (The Support)**
  - The automation and software (like CASE tools) that make the process and methods faster and more efficient.

Fig. - Software Engineering Layers

The software process

- A ***process*** is a collection of **activities, actions, and tasks** that are performed when some work product is to be created.
- A **Process Framework** is the essential structure of a software project, consisting of two parts:
  - **Framework Activities:** The core stages (like planning and coding) required for every project, no matter the size.
  - **Umbrella Activities:** Ongoing tasks (like risk management and quality tracking) that run throughout the entire project to keep it on track.

The software process

Framework activities

- These five framework activities form the **"Generic Process Model"** used to guide any software project from start to finish:
  - **Communication:** Collaborating with stakeholders to understand their goals and gather specific requirements.
  - **Planning:** Creating a "map" (the project plan) that defines tasks, risks, resources, and schedules.
  - **Modeling:** Building diagrams and designs to help everyone visualize the requirements and system structure.
  - **Construction:** The actual building phase—writing the code and performing tests to catch errors.
  - **Deployment:** Delivering the finished software to the customer for evaluation and feedback.

The software process

Umbrella Activities

- Are continuous tasks that run alongside the main development stages to ensure the project stays organized and high-quality.
  - **Project Tracking & Control**: Monitoring progress against the plan and adjusting stay on schedule.
  - **Risk Management**: Identifying and minimizing potential problems that could hurt the project or product quality.
  - **Quality Assurance (SQA):** Performing planned activities to guarantee the final software meets established standards.
  - **Technical Reviews**: Inspecting work (like code or designs) early to find and fix errors before they grow.

The software process

- **Measurement:** Collecting data and metrics to track how well the process, project, and product are performing.
- **Configuration Management (SCM):** Controlling and tracking changes made to the software and its documentation.
- **Reusability Management:** Finding ways to create and use components that can be shared across different projects.
- **Work Product Preparation:** The actual creation of necessary "paperwork," such as models, logs, and forms.

The software process

Attributes for Comparing Process Models

- **Overall flow and interdependencies among tasks**
  - How activities are ordered (sequential, iterative, parallel) and how they depend on each other.
- **Degree to which work tasks are defined**
  - Whether tasks are strictly specified (like in Waterfall) or loosely defined (like in Agile).
- **Degree to which work products are identified**
  - How clearly outputs (documents, code, models) are defined and required.
- **Manner of quality assurance activities**
  - How quality is maintained - formal reviews, continuous testing, or informal checks.

The software process

- **Project tracking and control approach**
  - How progress is monitored (milestones, daily standups, metrics, etc.).
- **Degree of detail and rigor**
  - Whether the process is heavyweight (strict, detailed) or lightweight (flexible, minimal documentation).
- **Stakeholder involvement**
  - How much users/customers are engaged throughout development.
- **Level of team autonomy**
  - Whether the team has freedom to make decisions or must strictly follow management directives.
- **Team organization and roles**
  - Whether roles are clearly defined (manager, tester, analyst) or more flexible and shared.

The Software Engineering Practice

- **The Essence of Practice**
  - Understand the problem (communication and analysis)
  - Plan a solution (software design)
  - Carry out the plan (code generation)
  - Examine the result for accuracy (testing and quality assurance)
- **Understand the Problem**
  - Who are the stakeholders?
  - What functions and features are required to solve the problem?
  - Is it possible to create smaller problems that are easier to understand?
  - Can a graphic analysis model be created?

The Software Engineering Practice

- **Plan the Solution**
  - Have you seen similar problems before?
  - Has a similar problem been solved?
  - Can readily solvable sub problems be defined?
  - Can a design model be created?
- **Carry Out the Plan**
  - Does solution conform to the plan?
  - Is each solution component provably correct?
- **Examine the Result**
  - Is it possible to test each component part of the solution?
  - Does the solution produce results that conform to the data, functions, and features required?

Software general principle

- The dictionary defines the word *principle* as “an important underlying law or assumption required in a system of thought.”
- **David Hooker** has Proposed **seven** principles that focus on software Engineering practice.
- **The First Principle: The Reason It All Exists**
  - A software system exists for one reason: to provide value to its users.
- **The Second Principle: KISS (Keep It Simple, Stupid!)**
  - Software design is not a haphazard process. There are many factors to consider in any design effort.
  - All design should be as simple a possible, but no simpler.

Software general principle

- **The Third Principle: Maintain the Vision**
  - A project must have a **clear and consistent goal**. Without a shared vision, the project can become confusing and inconsistent.
- **The Fourth Principle: What You Produce, Others Will Consume**
  - Software should be developed with the understanding that **other people will read, use, or maintain it**. So, it must be clear and understandable.
- **The Fifth Principle: Be Open to the Future**
  - Software should be designed to **adapt to future changes**. It should remain useful over time and continue to provide value to users.
- **The Sixth Principle: Plan for Reuse**
  - Designing software in a way that allows **reuse of components** helps save time, reduce cost, and increase overall efficiency.
- **The Seventh principle: Think!**
  - Careful thinking and planning before starting work usually led to **better and more accurate results**

Software myths

- Software myths are **incorrect beliefs about software and its development process.**
- They originated in the early days of computing when software development was **less structured and not well understood**.
- These myths continue to exist because:
  - They sound logical and easy to believe
  - They are often based on past experiences or assumptions
  - They are sometimes spread by experienced professionals who may rely on outdated practices
- However, these beliefs are **misleading** and can result in:
  - **Poor decision-making**
  - **Project delays**
  - **Increased cost**
  - **Low-quality software**

Management myths

- Under pressure of **time, cost, and quality**, they may look for **quick and easy solutions**.
- As a result, they sometimes believe in **software myths**, thinking they will solve problems.

Myth 1

We already have a complete book of standards and procedures for software development, so won't that be enough to guide the team?

- **Reality :**
  1. The book may exist, but is it used?
  2. Are software practitioners aware of its existence?
  3. Does it reflect modern software engineering practice?
  4. Is it complete?
  5. Is it adaptable?
  6. Is it streamlined to improve time to delivery while still maintaining a focus on Quality?
- In many cases, the answer to these entire question is NO.

Management myths

Myth 2

If we get behind schedule, we can add more programmers and catch up

- **Reality**
  - Software development is not like manufacturing where adding workers speeds up work.
  - In fact, adding more people to a delayed project can make it **even slower**.
  - This is because existing team members must spend time **training and supporting new members**.
  - As a result, less time is available for actual development, which delays the project further.

Management myths

Myth 3

If we outsource the software project to another company, we can relax and let them handle everything.

- **Reality**
  - Even when a project is outsourced, the organization must still **manage and monitor it properly**.
  - If the organization lacks internal project management skills, it will face **difficulties controlling the outsourced work**.

Customer myths

A customer who requests software can be:

- A person nearby
- A team within the organization
- A department like marketing or sales
- Or an external company

- Many customers believe software myths because developers and managers do not correct their misunderstandings.
- These false beliefs create unrealistic expectations.
- As a result, customers often become dissatisfied with the final product.

Customer myths

Myth 1

A general statement of objectives is sufficient to begin writing programs - we can fill in details later.

- **Reality**
  - Starting development with unclear or incomplete requirements can lead to serious problems.
  - Successful software development needs **clear, complete, and unambiguous requirements**.
  - These requirements are achieved only through **continuous communication between the customer and the developer**.

Customer myths

Myth 2

- **Software requirements keep changing, but changes are easy to handle because software is flexible**.
- **Reality**
  - It is true that requirements may change during a project.
  - However, the **impact of changes depends on the stage of the project**.
  - If changes are made early, they are easier and cheaper to handle.
  - If changes are made later, they become costly because:
    - Design is already completed
    - Resources are already used
    - Major rework may be required

Practitioner's myths

- Many software practitioners still follow myths developed over years of programming culture.
- Earlier, programming was seen as an art rather than an engineering discipline.
- Because of this, some outdated beliefs and practices still exist.
- Old habits in software development are difficult to change.

Myth 1

Once we write the program and get it to work, our job is done.

- **Reality**
  - Work is not finished when the program runs
  - Most work happens **after delivery**.
  - Around 60–80% effort is spent on **maintenance and improvements**.
  - So, coding is only a small part of the total work.

Practitioner's myths

Myth 2

Until I get the program "running" I have no way of assessing its quality.

- **Reality**
- Quality can be checked early using **formal technical reviews**.
- Reviews help find defects even before testing.
- They are more effective than testing in many cases.

Practitioner's myths

Myth 3

The only deliverable work product for a successful project is the working program.

- **Reality**
- A working program is only one part of software.
- Other important parts include:
  - Documentation
  - Design documents
  - Support materials
- These are important for maintenance and future use.

Practitioner's myths

Myth 4

Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.

- **Reality**
  - Software engineering focuses on **quality, not documents**.
  - Better quality reduces rework.
  - Less rework means **faster delivery in the long run**.

THE END OF A CHAPTER